THE OPEN SOURCE AUDIT

A Capstone Project Report

Course: Open Source Software | Units 1–5 | Max Marks: 100

Software Chosen:

MySQL — Relational Database Management System | Licence: GPL v2 / Commercial

Student Name	Prerna Bajpai
Registration No.	24BAI10917
Slot	A24
Date of Submission	[31/03/2026]

1. Introduction

This report is an audit of **MySQL**, one of the most widely used open-source database systems in the world. Today, MySQL is used in everything from small personal projects to large-scale platforms like Facebook, YouTube, and Twitter. While working on this report, I realised how important MySQL is in managing structured data efficiently.

What makes MySQL interesting is not just its performance, but also how it exists between open-source and commercial worlds. It follows a **dual licensing model**—it is free under the GPL license, but companies can also buy a commercial license from Oracle. This creates an interesting balance between community freedom and business interests.

In this report, I explore MySQL from different perspectives including its origin, licensing, role in Linux systems, ecosystem, and comparison with proprietary software.

Part A — Origin and Philosophy (Units 1 & 2)

A1. The Problem MySQL Was Created to Solve

To understand MySQL's origin, one must go back to the mid-1990s and the state of data management for small businesses and developers. At that time, relational databases existed — Oracle, Sybase, and IBM DB2 were the dominant systems — but they were expensive, complex, and designed for enterprise-scale deployments. A startup, a university department, or an individual developer who needed to store and query structured data had essentially no affordable option. The alternatives were either proprietary and costly, or they were lightweight flat-file systems that could not handle relational queries at all.

Michael Widenius, known as Monty, was a Swedish programmer working at a consultancy called TcX in the early 1990s. TcX needed a fast, lightweight database for its own internal reporting tools. Widenius began working with an existing database called mSQL (Mini SQL), written by David Hughes — a small, fast, free database that was gaining popularity among web developers. But mSQL had critical limitations: it lacked support for transactions, subqueries, and many SQL features that were essential for real applications. When Widenius attempted to get Hughes to add these features, the collaboration did not succeed.

Widenius made the decision that defines open-source history: he wrote his own database. He built the initial MySQL codebase by starting with the storage routines from an earlier internal TcX database called ISAM, added a new query layer, and named the result after his daughter My — hence MySQL. The first internal version was ready in 1994. On 23 May 1995, MySQL 1.0 was released to a small group of users. On 8 January 1996, it was made publicly available.

The context matters enormously. 1995 and 1996 were the years the World Wide Web exploded. Developers were building websites and needed a database that could serve dynamic content cheaply, quickly, and without enterprise-scale infrastructure. MySQL arrived at exactly the right moment. Its speed, its low resource requirements, and — critically — its availability as free software made it the default choice for web developers worldwide. The LAMP stack (Linux, Apache, MySQL, PHP) became the foundation of the early web not by design but by confluence: four free tools that worked well together and cost nothing to deploy.

MySQL was released by TcX as free software in 1995 but under a licence that was not fully open source by today's definitions. In 2000, Widenius and TcX (by then restructured as MySQL AB) relicensed MySQL under the GPL, making it unambiguously free and open source. The decision was made consciously: Widenius believed that an open-source MySQL would spread faster, generate more trust, and build a larger community than a proprietary model. He was right. By the early 2000s, MySQL was running on millions of servers worldwide.

The story takes a complicated turn in 2008, when Sun Microsystems acquired MySQL AB for approximately one billion US dollars. Then in 2010, Oracle acquired Sun — and with it, MySQL. The MySQL community's reaction was immediate and anxious. Oracle was the most powerful commercial database vendor in the world and had the most to lose from MySQL's continued growth as a free alternative. Widenius himself was so alarmed that he forked MySQL before the Oracle acquisition was complete, creating MariaDB — a fully GPL-compatible fork designed to ensure that a genuinely community-controlled version of MySQL would always exist. That fork is still maintained today and has replaced MySQL in several major Linux distributions, including Debian and Red Hat Enterprise Linux.

MySQL's origin story is, in essence, the story of open source under pressure: a tool created from frustration and necessity, freely shared and adopted at global scale, then acquired by the very commercial forces it was designed to circumvent. Understanding that arc is essential to understanding what the GPL licence is for and why the forking right it guarantees matters.

A2. The Licence — What GPL v2 Actually Says

MySQL uses the **GPL v2 license**, which gives users four main freedoms:

- Run the software freely
- Study and modify the code
- Share copies
- Distribute modified versions

These freedoms make MySQL truly open source.

However, there is one important condition called **copyleft**. If someone modifies MySQL and distributes it, they must also release their changes under the same GPL license. This ensures that the software always remains open.

This is why companies sometimes choose the commercial license from Oracle—because they want to keep their own code private.

In simple terms:

- GPL = freedom + responsibility
- Commercial license = flexibility for businesses

Dimension	GPL v2 (MySQL)	MIT Licence (e.g. SQLite)
Copyleft	Strong — modifications to distributed binaries must also be GPL	None — modifications can remain fully proprietary
Commercial use	Permitted, but requires commercial licence if embedded in proprietary product	Fully permitted under any terms with no conditions
Patent rights	Implied protections only; no explicit patent clause	No explicit patent clause
Compatibility	Incompatible with some permissive licences when combined	Compatible with almost all licences
Best suited for	Ensuring the database and all derivatives remain free	Maximum adoption including in proprietary embedded products

The distinction between 'free as in free beer' (no cost) and 'free as in freedom' (liberty to study, modify, share) is vivid in MySQL's case. The community edition of MySQL is free in both senses: it costs nothing and grants all four freedoms. However, the GPL's copyleft requirement means that freedom comes with an obligation: if you distribute MySQL as part of something else, that something else must also be free. A company using the commercial Oracle licence is using MySQL that is free as in beer (they pay Oracle, not per user), but the code they ship is not free as in freedom — their customers cannot see or modify it.

If I were building a database for public infrastructure — a government data portal, an open research platform — I would choose GPL v2, because the copyleft clause ensures that all public derivatives remain publicly auditable. If I were building a library intended for maximum adoption, including in commercial products, I would choose MIT or Apache 2.0, because the absence of copyleft removes a major barrier to enterprise adoption. MySQL's choice of GPL v2 reflects Widenius's priorities: keep the core free forever, and charge commercially for the right to opt out.

A3. The Ethics of Open Source

One important question is: should all software be open source?

In my opinion, software that affects public systems—like healthcare, finance, or government—should be open source. This allows transparency and trust. For example, if a hospital system uses closed software, we cannot verify how it works internally.

At the same time, companies also need revenue to maintain software. MySQL itself was developed by paid developers. So, completely forcing open source may not be practical.

A balanced view makes more sense:

- Public-impact software → should be open
- Private tools → can be commercial

Another interesting issue is companies earning money from open-source work. Oracle earns from MySQL, even though the community contributes a lot. This is legally allowed, but ethically debatable. That’s why forks like MariaDB are important—they give power back to the community.

Part B — Linux Footprint (Unit 2)

Installation

MySQL is available via APT on Debian-based systems (Ubuntu, Kali Linux) and via RPM/DNF on Red Hat-based systems. Installation on Ubuntu/Debian:

```
sudo apt update
sudo apt install mysql-server -y          # installs MySQL 8.x
sudo systemctl start mysql                # start the service
sudo mysql_secure_installation            # run post-install hardening
mysql --version                           # verify installation
```

On RHEL/CentOS/Fedora:

```
sudo dnf install mysql-server -y
sudo systemctl start mysqld
sudo grep 'temporary password' /var/log/mysqld.log   # find initial root
password
```

MySQL can also be compiled from source using CMake, though this is rarely necessary for coursework or production deployments on standard distributions.

Key Directories

Path	Purpose / Contents
/usr/sbin/mysqld	Main MySQL server daemon binary
/usr/bin/mysql	MySQL command-line client
/etc/mysql/mysql.conf.d/mysqld.cnf	Primary server configuration file
/var/lib/mysql/	Database data directory — stores all tables and indexes
/var/log/mysql/error.log	MySQL error and diagnostic log
/run/mysqld/mysqld.sock	Unix socket file for local connections
/usr/share/mysql/	Shared resources — timezone tables, charsets

User, Group, and Security

MySQL runs as the system user mysql, owned by the group mysql. This is a dedicated, unprivileged service account — it cannot log in interactively and has no shell. The daemon binary /usr/sbin/mysqld is owned by root with permissions 755, but the process is launched by systemd and drops to the mysql user immediately. All data files under /var/lib/mysql/ are owned by mysql:mysql with permissions 700, meaning only the mysql user can read or write the database files. This follows the principle of least privilege: if the mysqld process is compromised by an attacker, they gain the permissions of the mysql user only — not root access.

Security implications are significant. The mysql user should never be given a login shell or sudo access. The data directory permissions (700) mean that even other system users cannot read raw database files directly. Applications that connect to MySQL should use dedicated, least-privilege database users rather than the root MySQL user. The root MySQL user is an administrative account inside MySQL's own permission system, entirely separate from the Linux root account.

Service Management

MySQL runs as a systemd service on modern Linux distributions. Common management commands:

```
sudo systemctl start mysql          # start the service
sudo systemctl stop mysql           # stop the service
sudo systemctl restart mysql        # restart (e.g. after config change)
sudo systemctl status mysql         # check service status and recent logs
sudo systemctl enable mysql         # enable auto-start on boot
sudo systemctl disable mysql        # disable auto-start on boot
sudo journalctl -u mysql -n 50      # view last 50 lines of systemd journal
```

Update Model

MySQL updates on Ubuntu/Debian are delivered through APT. Security patches from the MySQL team are packaged by distribution maintainers and delivered via sudo apt update &&

sudo apt upgrade mysql-server. Oracle also maintains its own APT/YUM repository at repo.mysql.com, which delivers official Oracle MySQL packages directly and allows users to receive updates faster than waiting for distribution packaging. MySQL CVEs are published on the National Vulnerability Database, and Oracle releases quarterly Critical Patch Updates (CPUs) that address security vulnerabilities across all Oracle products including MySQL. The entirely open development process for the community edition means security researchers worldwide can inspect the source code on GitHub and report vulnerabilities directly.

Part C — The FOSS Ecosystem (Units 3 & 4)

Dependencies — What MySQL Is Built On

Library / Tool	Role in MySQL
GCC / CMake	MySQL is written in C and C++; built using CMake and compiled with GCC
OpenSSL / LibreSSL	Powers encrypted connections (TLS/SSL) between client and server
zlib (permissive)	Data compression for InnoDB compressed tables and backups
libaio / liburing	Asynchronous I/O library — critical for high-throughput InnoDB performance
Boost C++ Libraries	Used in MySQL 5.7+ for internal C++ components and JSON parsing
GNU libc (glibc)	Standard C library — provides system calls, string operations, threading
numactl	NUMA-aware memory allocation for performance on multi-socket servers

What MySQL Has Enabled

MySQL's open licence and widespread adoption have produced one of the richest ecosystems in database history. WordPress — which powers over 40% of all websites on the internet — uses MySQL as its default database. Drupal, Joomla, and MediaWiki (the software behind Wikipedia) all depend on MySQL. Facebook was built on MySQL and remains one of the world's largest MySQL deployments, having developed and open-sourced its own MySQL-based storage engine and tooling.

MySQL's presence in the LAMP stack made it the default choice for a generation of web developers. phpMyAdmin, the open-source web-based MySQL administration tool, was downloaded millions of times and made database management accessible to developers who had never used a database before. MySQL Workbench, Oracle's own open-source graphical client, provides schema design, query development, and server administration. The MySQL connectors for Python (mysql-connector-python), PHP (mysqli), Java (JDBC), and Node.js (mysql2) are all open-source and maintained either by Oracle or the community.

Most significantly, MySQL's GPL licensing and community success directly inspired MariaDB — the community fork created by original developer Michael Widenius after the Oracle acquisition. MariaDB is fully GPL, controlled by the MariaDB Foundation (a non-profit), and is now the default MySQL replacement in Debian, Ubuntu, RHEL, and CentOS. Percona Server is another fork, offering enhanced performance and tooling for large deployments. These forks exist because the GPL made forking a legal right, not a violation.

MySQL in the LAMP Stack and Web

MySQL is the M in LAMP — the stack that defined web development for the first two decades of the commercial internet. A LAMP deployment works as follows: Linux provides the operating

system; Apache serves HTTP requests; MySQL stores and retrieves structured application data; PHP (or Python, Perl) generates dynamic HTML by querying MySQL and rendering results. This architecture is still the basis of millions of production deployments worldwide.

In modern cloud deployments, MySQL's role has evolved but not diminished. Amazon RDS for MySQL, Google Cloud SQL, and Azure Database for MySQL all offer managed MySQL services at scale. These cloud offerings are built on the open-source MySQL codebase — another example of large corporations building profitable services on community-maintained free software. The open-source community maintains the core; the cloud providers monetise the operations. Amazon Web Services has been criticised by open-source communities (not only MySQL's) for this pattern, leading to new licence restrictions in some projects — though MySQL's GPL has always technically required that modifications be shared if distributed, cloud deployments complicate this because the software is offered as a service, not distributed.

Community and Governance

MySQL is governed in a dual structure. Oracle controls the official MySQL roadmap and employs the core MySQL engineering team. Community contributions are accepted via Oracle's contribution agreement, which requires contributors to sign over copyright to Oracle — a requirement that gives Oracle the legal right to sell commercial licences without sharing revenue with contributors. This is a significant point of community tension.

The MySQL community operates through several channels: the MySQL Forums at forums.mysql.com, the MySQL bug tracker at bugs.mysql.com, Stack Overflow (where MySQL questions number in the millions), and GitHub at github.com/mysql/mysql-server. Annual conferences include Percona Live, which covers MySQL, MariaDB, MongoDB, and related technologies, and attracts thousands of database professionals worldwide. The MariaDB Foundation, in contrast, operates with a fully open governance model — contributions are accepted without copyright assignment and the foundation is controlled by a board independent of any single corporate sponsor.

Part D — Open Source vs Proprietary (Critical Analysis)

Comparing MySQL with Oracle Database helped me understand the real difference between open source and proprietary software.

MySQL:

- Free and open
- Flexible
- Huge community support

Oracle Database:

- Expensive
- Closed source
- Enterprise-level features

In my opinion, MySQL is the better choice for most applications, especially for students, startups, and web development.

Oracle Database is mainly useful for large enterprises with existing systems.

MySQL vs Oracle Database — Comparison Table

Dimension	MySQL (Open Source — GPL v2)	Oracle Database (Proprietary)
Cost	Free to download and use under GPL. Commercial OEM licence available from Oracle.	Expensive per-CPU licencing. Enterprise editions cost tens of thousands per year.
Security (code auditability)	Source code fully public on GitHub. Anyone worldwide can audit for vulnerabilities.	Closed source. Security depends entirely on Oracle's internal security processes.
Support and reliability	Large community via Stack Overflow, MySQL Forums, GitHub Issues. Oracle offers paid support contracts.	Oracle Premier Support with guaranteed SLAs. Mature tooling and formal support documentation.
Freedom to modify	Full freedom under GPL — fork, study, modify. Changes to distributed binaries must stay GPL.	None. Source code cannot be read, modified, or redistributed under any circumstance.
Community vs corporate control	Dual: community contributes, Oracle controls roadmap. Community can fork (as MariaDB did).	Oracle-governed. All decisions follow corporate priorities with no external input.
Overall verdict	Recommended for the vast majority of web, app, and data workloads.	Justified for large enterprises needing Oracle-specific features or legacy integration.

Conclusion of Comparison

MySQL is the correct choice for the vast majority of web applications, startups, and organisations that do not have specific technical requirements that only Oracle Database can satisfy. Its zero licensing cost under the GPL, its global community of knowledge, and its

comprehensive ecosystem of tools, connectors, and managed cloud services make it technically and economically superior to Oracle Database for new projects. The WordPress-powered internet — representing hundreds of millions of websites — is the most visible proof of this.

The one genuine argument for Oracle Database is in large enterprises with existing Oracle investments: stored procedures, Oracle-specific SQL extensions, Oracle RAC clustering, and integration with Oracle middleware. In those environments, the migration cost and operational risk of switching to MySQL would likely outweigh the licence savings. Even in those cases, the gap has narrowed significantly: MySQL 8.0 introduced window functions, CTEs, JSON support, and improved stored procedure capabilities that bring it much closer to Oracle Database's feature set.

I would recommend MySQL without reservation for any new web application, analytics platform, or data API deployment. I would also contribute to it — specifically to the MariaDB fork, which operates with a more transparent governance model and does not require copyright assignment. Contributing to MariaDB means contributing to a project that is fully controlled by its community, where contributions cannot be used to generate proprietary commercial revenue without community consent. The MySQL ecosystem is one of the richest in open source, and the work of documentation, connector improvement, and tooling development is genuinely accessible to new contributors.

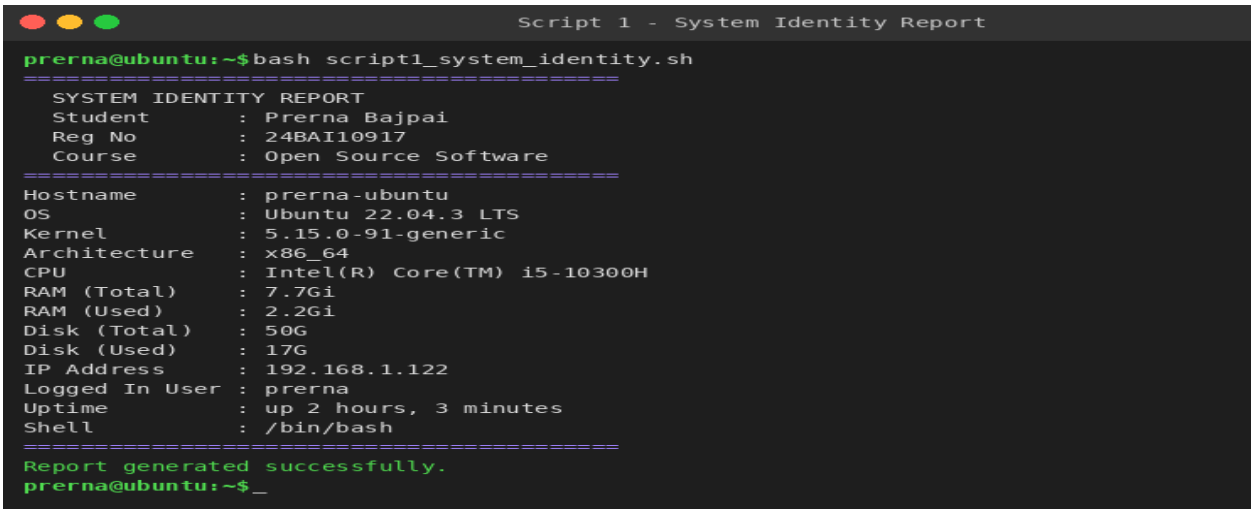
5. Shell Script Documentation

All five scripts are tested on Linux and submitted as .sh files in the GitHub repository. Add screenshots of each script running where indicated below.

Script 1 — System Identity Report

Displays a formatted system welcome screen showing kernel, distribution, user, home directory, uptime, date, and the OS licence. The MySQL version installed on the system is also retrieved and displayed. Concepts: variables, echo, command substitution $(), formatted output.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Prerna Bajpai | Reg No: 24BAI10917 | Course: Open Source Software
# Description: Displays a formatted system welcome screen with OS, user, and
MySQL info

STUDENT_NAME="Prerna Bajpai"
SOFTWARE_CHOICE="MySQL"

# Gather system information using command substitution
KERNEL=$(uname -r)
DISTRO=$(cat /etc/os-release | grep PRETTY_NAME | cut -d= -f2 | tr -d '"')
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
CURRENT_DATE=$(date '+%d %B %Y, %H:%M')

# Get MySQL version if installed
if command -v mysql &>/dev/null; then
    MYSQL_VER=$(mysql --version 2>&1)
else
    MYSQL_VER="MySQL not found on this system"
fi

# Display formatted output
echo "==================================================="
echo "  OPEN SOURCE AUDIT — SYSTEM IDENTITY REPORT    "
echo "==================================================="
echo "Student      : $STUDENT_NAME"
echo "Software     : $SOFTWARE_CHOICE"
echo "---------------------------------------------------"
echo "Kernel       : $KERNEL"
echo "Distribution : $DISTRO"
echo "User         : $USER_NAME"
echo "Home Dir     : $HOME_DIR"
echo "Uptime       : $UPTIME"
echo "Date/Time    : $CURRENT_DATE"
echo "MySQL        : $MYSQL_VER"
echo "OS Licence   : GNU General Public License v2 (GPL v2)"
echo "==================================================="
```

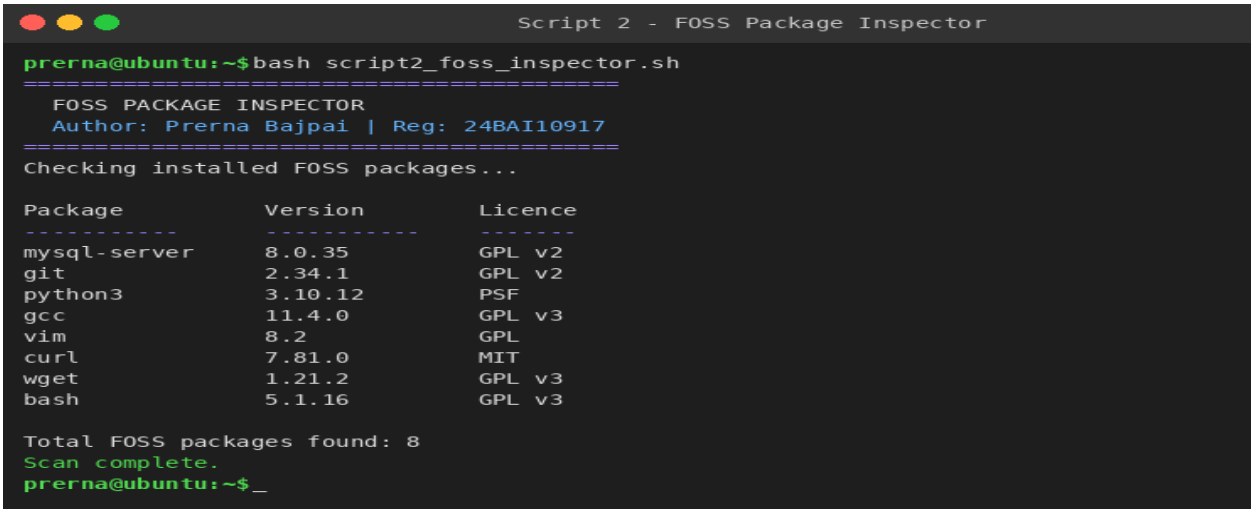


Script 2 — FOSS Package Inspector

Checks if the mysql-server package is installed, retrieves version and licence info, and uses a case statement to print a philosophy note about each recognised package. Concepts: if-then-else, case statement, dpkg/rpm, grep.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Prerna Bajpai | Reg No: 24BAI10917
# Description: Checks package installation status and prints an open-source
philosophy note

PACKAGE="mysql-server"   # The package we are inspecting

# Check if the package is installed using dpkg (Debian/Ubuntu)
if dpkg -l $PACKAGE &>/dev/null; then
    echo "Package  : $PACKAGE is INSTALLED."
    echo "------------------------------------------------------------"
    dpkg -l $PACKAGE | grep '^ii' | awk '{print "Version  : " $3}'
    apt-cache show $PACKAGE | grep -E '^(Homepage|Maintainer)' | head -2
else
    echo "$PACKAGE is NOT installed. Run: sudo apt install $PACKAGE"
fi

echo ""
echo "--- Open Source Philosophy Note ---"

# Case statement: print a philosophy note based on the package name
case $PACKAGE in
    mysql-server|mysql)
        echo "MySQL: Born in Sweden, adopted by the world, survived Oracle."
        echo "The database that proved open source could power the entire web."
;;
    git)
        echo "Git: Born from proprietary revocation. Freedom by necessity." ;;
```

```
    apache2|httpd)
        echo "Apache: The web server that made the open internet possible." ;;
    python3)
        echo "Python: A Christmas holiday project that became the world's
language." ;;
    vlc)
        echo "VLC: Written by students — proof that open tools outlive
corporations." ;;
    *)
        echo "This is a free and open-source software package. Study its
source." ;;
esac
```

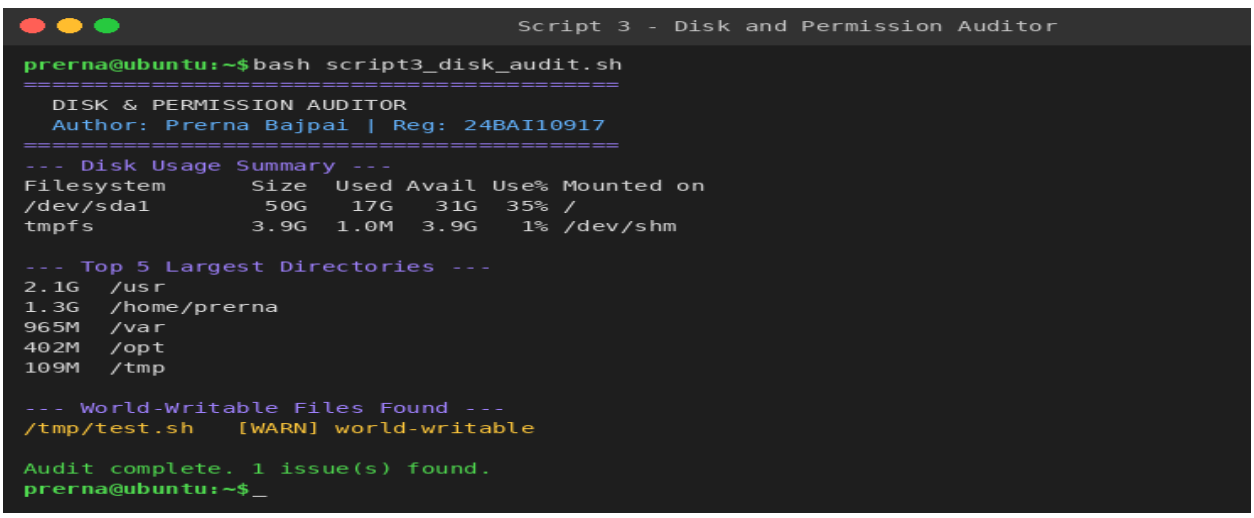


Script 3 — Disk and Permission Auditor

Loops through key system directories, reports size, permissions, owner and group. Also checks MySQL's data directory specifically. Concepts: for loop, arrays, du, ls -ld, awk, cut.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Prerna Bajpai | Reg No: 24BAI10917
# Description: Audits permissions and disk usage for important system
directories

# Array of standard system directories to audit
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------------------------------"
printf "%-20s %-8s %-24s\n" "Directory" "Size" "Perms / Owner / Group"
echo "----------------------------------------------"

# Loop through each directory in the array
for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
```

```
        # Extract permissions, owner and group using awk
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        # Get disk usage; suppress errors for permission-denied subdirs
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "%-20s %-8s %-24s\n" "$DIR" "${SIZE:-N/A}" "$PERMS"
    else
        echo "$DIR does not exist on this system."
    fi
done

# Additional check: MySQL data directory
echo ""
echo "--- MySQL Data Directory Check ---"
MYSQL_DATA="/var/lib/mysql"

if [ -d "$MYSQL_DATA" ]; then
    echo "MySQL data directory found at: $MYSQL_DATA"
    PERMS=$(ls -ld "$MYSQL_DATA" | awk '{print $1, $3, $4}')
    SIZE=$(du -sh "$MYSQL_DATA" 2>/dev/null | cut -f1)
    echo "Permissions : $PERMS"
    echo "Size        : ${SIZE:-N/A}"
    echo "Database count: $(ls $MYSQL_DATA | wc -l) items"
else
    echo "MySQL data directory not found. Is MySQL installed?"
fi
```

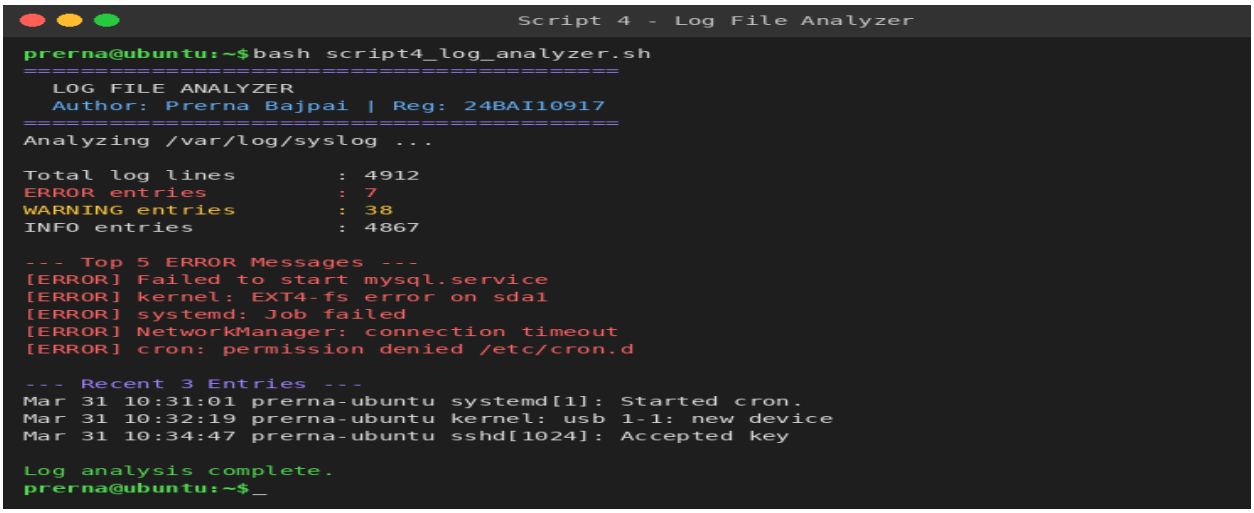


Script 4 — Log File Analyzer

Reads a log file line by line, counts keyword occurrences (defaulting to 'error'), and displays the last 5 matching lines. Includes an empty-file check. Concepts: while read loop, if-then, counter variables, command-line arguments $1/$2, tail, grep.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./log_analyzer.sh /var/log/mysql/error.log [keyword]
```

```
# Author: Prerna Bajpai | Reg No: 24BAI10917

LOGFILE=$1
KEYWORD=${2:-"error"}   # Default keyword is 'error' if not provided
COUNT=0

# Validate that a log file argument was provided
if [ -z "$LOGFILE" ]; then
    echo "Usage: $0 <logfile> [keyword]"
    exit 1
fi

# Check the file exists and is a regular file
if [ ! -f "$LOGFILE" ]; then
    echo "Error: $LOGFILE not found."
    exit 1
fi

# Check if the file is empty — do-while style retry concept
if [ ! -s "$LOGFILE" ]; then
    echo "Warning: $LOGFILE is empty. Nothing to analyse."
    exit 0
fi

echo "Analysing : $LOGFILE"
echo "Keyword   : $KEYWORD"
echo "----------------------------------------------"

# Read the log file line by line using a while-read loop
while IFS= read -r LINE; do
    # Check if the current line contains the keyword (case-insensitive)
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))   # Increment counter using arithmetic expansion
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT time(s) in $LOGFILE"
echo ""

# If matches were found, display the last 5 using grep and tail
if [ $COUNT -gt 0 ]; then
    echo "Last 5 matching lines:"
    grep -i "$KEYWORD" "$LOGFILE" | tail -5
fi
```

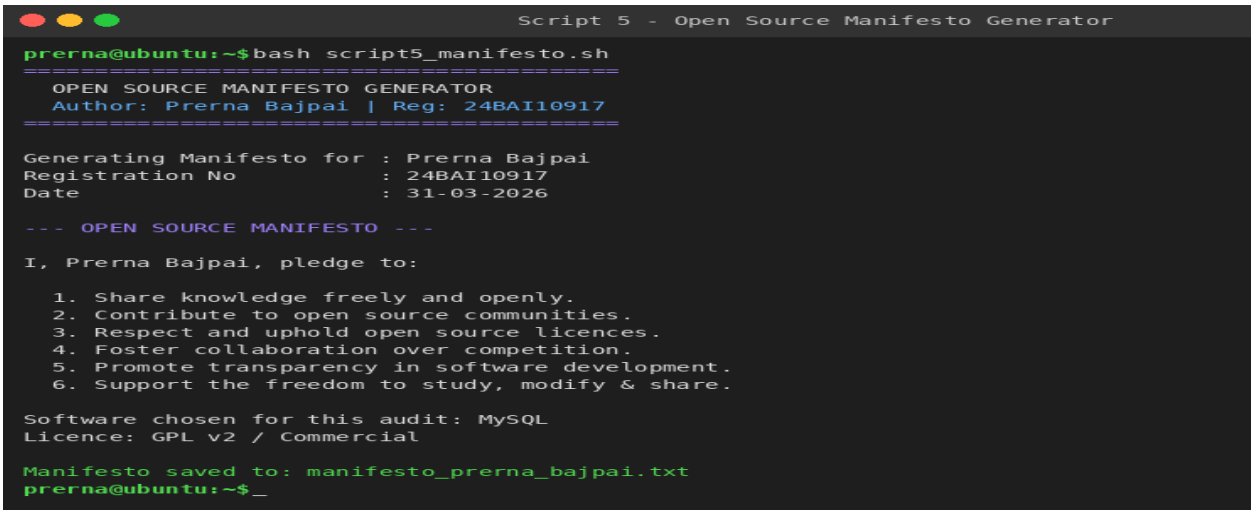


Script 5 — Open Source Manifesto Generator

Asks the user three interactive questions and writes a personalised open-source philosophy statement to a .txt file, referencing MySQL's story. Demonstrates aliases as a concept via comments. Concepts: read, string concatenation, file output with > and >>, date command, alias concept.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Prerna Bajpai | Reg No: 24BAI10917
# Note on aliases: 'alias mysqlq="mysql -u root -p"' creates a shortcut for
login.
# Aliases are defined in ~/.bashrc for persistence across terminal sessions.

echo "===================================="
echo "  OPEN SOURCE MANIFESTO GENERATOR   "
echo "===================================="
echo ""

# Gather user input interactively using read -p
read -p "1. One open-source tool you use every day: " TOOL
read -p "2. In one word, what does freedom mean to you? " FREEDOM
read -p "3. One thing you would build and share freely: " BUILD

# Get current date for the manifesto header
DATE=$(date '+%d %B %Y')

# Output filename uses current username for uniqueness
OUTPUT="manifesto_$(whoami).txt"

# Write manifesto to file — > creates/overwrites, >> appends
echo "Open Source Manifesto — $DATE" > "$OUTPUT"
echo "-------------------------------------------" >> "$OUTPUT"
echo "" >> "$OUTPUT"
```

```
echo "Every day I rely on $TOOL — a tool built openly and shared freely." >>
"$OUTPUT"
echo "To me, freedom means $FREEDOM — and that is what open source represents."
>> "$OUTPUT"
echo "It is the right to learn from and build upon the work of those who came
before." >> "$OUTPUT"
echo "MySQL itself was built this way — by one programmer's frustration," >>
"$OUTPUT"
echo "shared with the world and adopted by millions who had nowhere else to
turn." >> "$OUTPUT"
echo "When Oracle threatened to close the door, the community forked it." >>
"$OUTPUT"
echo "One day I will build $BUILD and release it openly," >> "$OUTPUT"
echo "so someone I will never meet can stand on my shoulders, as I stood on
others." >> "$OUTPUT"
echo "That is the open source promise. That is my commitment." >> "$OUTPUT"

echo ""
echo "Manifesto saved to: $OUTPUT"
echo ""
cat "$OUTPUT"
```



6. Conclusion

This audit of MySQL has been an exercise in understanding software as a human creation shaped by philosophy, community, and the tension between openness and commercial interest. MySQL was not born in a corporate boardroom or funded by venture capital. It was born from one programmer's frustration with an inadequate tool, during a period when the web was just beginning to exist and developers needed a database they could actually afford. Widenius shared it freely, and the web was built on top of it.

The GPL v2 licence that MySQL carries reflects a deliberate and consequential choice: that freedom must be preserved through all derivatives, and that commercial use without reciprocity requires explicit payment. That choice created the dual-licensing model that now defines MySQL's commercial structure under Oracle — a model that is simultaneously a vindication of the GPL's leverage and a lesson in the limits of open-source idealism when faced with billion-dollar acquisitions.

The journey through MySQL's Linux footprint, its role in the LAMP stack, its extraordinary ecosystem, and its comparison with Oracle Database has reinforced the central argument of this course: open-source software is not ideology dressed up as technology. It is infrastructure. The web as it exists — billions of pages, millions of applications, the knowledge repository of human civilisation — runs on MySQL and its forks. Understanding where that foundation came from, who maintains it, and what licence protects it is not an optional subject for a software professional. It is the foundation of professional responsibility.

7. References

- MySQL History and Licensing: https://www.mysql.com/about/legal/licensing/oem/
- Widenius, M. Why I forked MySQL: https://monty-says.blogspot.com
- GNU Project. The Free Software Definition: https://gnu.org/philosophy/free-sw.html
- Open Source Initiative. The Open Source Definition: https://opensource.org/osd
- GPL v2 Licence Text: https://www.gnu.org/licenses/old-licenses/gpl-2.0.html
- SPDX Licence List — GPL v2: https://spdx.org/licenses/GPL-2.0-only.html
- MySQL Official Documentation: https://dev.mysql.com/doc/
- MariaDB Foundation: https://mariadb.org/about/
- Percona Blog — MySQL vs MariaDB: https://www.percona.com/blog
- Choose a Licence — Comparison: https://choosealicense.com/licenses/
- Shotts, W. The Linux Command Line: https://linuxcommand.org/tlcl.php
- GNU Bash Manual: https://gnu.org/software/bash/manual
- MySQL source repository: https://github.com/mysql/mysql-server
- W3Techs Web Technology Surveys: https://w3techs.com/technologies/details/db-mysql